

# Relatório de Laboratório

---

*Laboratório de Experimentação de Software — Lab01S02*

|                                              |                                                                                                                                                                                                                                                                  |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Curso</b>                                 | Engenharia de Software                                                                                                                                                                                                                                           |
| <b>Disciplina</b>                            | Laboratório de Experimentação de Software                                                                                                                                                                                                                        |
| <b>Turno / Período</b>                       | Noite / 6º                                                                                                                                                                                                                                                       |
| <b>Professor(a)</b>                          | Danilo Maia                                                                                                                                                                                                                                                      |
| <b>Laboratório</b>                           | Lab01 — Análise dos repositórios mais populares do GitHub                                                                                                                                                                                                        |
| <b>Grupo (trio)</b>                          | Diogo Caribe, Pedro Alves                                                                                                                                                                                                                                        |
| <b>Link do repositório / GitHub Projects</b> | <a href="https://github.com/PedroMaiaAlves/lab-experimentacao-software">https://github.com/PedroMaiaAlves/lab-experimentacao-software</a><br><a href="https://github.com/users/PedroMaiaAlves/projects/1">https://github.com/users/PedroMaiaAlves/projects/1</a> |
| <b>Data de entrega</b>                       | 20/08/2026                                                                                                                                                                                                                                                       |

# 1. Introdução

O GitHub é uma plataforma amplamente utilizada para hospedagem, versionamento e colaboração em projetos de software. Seus repositórios apresentam diferentes níveis de popularidade, maturidade, participação da comunidade e atividade de desenvolvimento. A mineração dessas informações permite observar características recorrentes em projetos que alcançaram grande visibilidade e compreender como esses sistemas evoluem, recebem contribuições e administram suas atividades.

Neste laboratório foi realizada uma análise exploratória dos 1.000 repositórios públicos com maior número de estrelas retornados pela API GraphQL do GitHub. O número de estrelas foi adotado como definição operacional de popularidade. Após a coleta, os dados foram transformados em formato tabular, exportados para CSV e analisados de acordo com sete questões de pesquisa propostas pelo professor.

O objetivo geral do trabalho é caracterizar os repositórios populares do GitHub com base em indicadores de idade, pull requests mescladas, releases, recência de atualização, linguagem primária e percentual de issues fechadas. Também foi investigada a relação entre a popularidade da linguagem e os indicadores de contribuição, releases e atualização.

Além das atividades solicitadas, o grupo desenvolveu uma aplicação web em Streamlit. Essa aplicação permite iniciar a mineração por meio de uma interface gráfica, acompanhar o processamento, visualizar os resultados das questões de pesquisa e exportar os dados. A utilização do Streamlit representa a inovação proposta pelo grupo, pois as sete questões de pesquisa foram fornecidas pelo professor.

As hipóteses apresentadas neste relatório possuem caráter informal e exploratório. Elas são utilizadas para orientar a interpretação descritiva dos resultados, sem representar hipóteses confirmatórias ou pré-registradas.

| Questão de pesquisa                                                   | Hipótese informal                                                                                                                                                                                                             |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RQ01 — Sistemas populares são maduros/antigos?                        | Espera-se que os repositórios populares sejam majoritariamente projetos maduros, apresentando idade mediana superior a cinco anos.                                                                                            |
| RQ02 — Sistemas populares recebem muita contribuição externa?         | Espera-se que os repositórios populares acumulem um número elevado de pull requests mescladas, com mediana superior a 500, e apresentem uma distribuição assimétrica, na qual poucos projetos concentram valores muito altos. |
| RQ03 — Sistemas populares lançam releases com frequência?             | Espera-se que a maioria dos repositórios populares possua ao menos uma release publicada no GitHub e acumule múltiplas releases ao longo de sua existência.                                                                   |
| RQ04 — Sistemas populares são atualizados com frequência?             | Espera-se que a maioria dos repositórios populares apresente atividade recente, com o último push ocorrido nos últimos 30 dias, embora exista uma parcela de projetos inativos há períodos prolongados.                       |
| RQ05 — Sistemas populares são escritos nas linguagens mais populares? | Espera-se que a maioria dos repositórios populares com linguagem primária definida utilize uma das dez linguagens mais populares do GitHub Octoverse 2025.                                                                    |

| Questão de pesquisa                                                                       | Hipótese informal                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RQ06 — Sistemas populares possuem alto percentual de issues fechadas?                     | Espera-se que os repositórios populares que possuem issues apresentem percentual mediano de fechamento superior a 80%.                                                                                                                                                         |
| RQ07 — Linguagens populares recebem mais contribuição, mais releases e mais atualizações? | Espera-se que repositórios cuja linguagem primária pertence ao Top 10 do GitHub Octoverse apresentem maior mediana de pull requests mescladas e releases, além de menor tempo desde o último push, quando comparados aos repositórios escritos em outras linguagens definidas. |

## 2. Contexto

O trabalho foi desenvolvido durante as sprints S01 e S02 do Lab01 da disciplina de Laboratório de Experimentação de Software. Na S01 foram implementadas a consulta inicial à API GraphQL, a transformação dos dados e as análises associadas às questões de pesquisa. Também foi criada a primeira versão da interface web em Streamlit.

Na S02, a coleta foi ampliada para os 1.000 repositórios exigidos pelo laboratório. Para isso, foi utilizada paginação baseada em cursores. Também foram implementadas a exportação para CSV, a validação dos dados das sete RQs, a criação das hipóteses informais, a evolução dos gráficos, a atualização da metodologia e a preparação do primeiro snapshot do GitHub Project.

O objeto de estudo é composto pelos 1.000 repositórios públicos retornados pela consulta:

```
stars:>0 is:public sort:stars-desc
```

Essa expressão restringe a busca a repositórios públicos e solicita a ordenação decrescente pelo número de estrelas. A escolha das estrelas oferece um critério objetivo de seleção, mas não significa que popularidade represente diretamente qualidade, número de usuários ou utilização em produção.

A conexão GraphQL search retorna no máximo 1.000 resultados. Além disso, as conexões da API exigem paginação e aceitam no máximo 100 elementos em cada argumento first ou last. O grupo utilizou lotes de até 50 repositórios e percorreu as páginas por meio de endCursor e hasNextPage, conforme o mecanismo oficial de paginação do GitHub (GITHUB, 2026a; GITHUB, 2026b).

A literatura sobre mineração de repositórios também alerta que dados obtidos do GitHub precisam ser interpretados considerando limitações de seleção, representação e utilização dos recursos da plataforma. Nem todos os repositórios utilizam pull requests, releases ou issues da mesma forma, o que pode afetar a interpretação das métricas (KALLIAMVAKOU et al., 2014).

Para a RQ05 e a RQ07, foi adotado o ranking do GitHub Octoverse 2025. As dez linguagens utilizadas como referência foram TypeScript, Python, JavaScript, Java, C#, PHP, Shell, C++, HCL e Go. O ranking é baseado na quantidade de contribuidores observada pelo GitHub e representa um retrato temporal da atividade na plataforma (GITHUB, 2025).

## 3. Metodologia

Este estudo possui natureza quantitativa, observacional e exploratória. Foram utilizadas informações públicas obtidas diretamente da API GraphQL do GitHub. O processo foi dividido em planejamento, coleta, paginação, transformação, validação, análise, visualização e exportação.

### 3.1 Principais Desafios

O primeiro desafio foi coletar os 1.000 repositórios. Uma requisição GraphQL não retorna toda essa quantidade de uma só vez, tornando necessária a implementação de paginação baseada em cursores. O coletor também precisava verificar se existia uma próxima página e se o cursor retornado realmente permitia avançar.

Outro desafio foi garantir a cardinalidade e a unicidade da amostra. Um coletor que simplesmente execute determinado número de requisições pode terminar com menos registros do que o solicitado. Por isso, a implementação do `main.py` acompanha a quantidade efetivamente coletada, elimina nomes repetidos e somente conclui quando alcança 1.000 repositórios únicos.

A consistência entre os formatos também foi relevante. Os dados extraídos precisavam gerar uma lista de nós GraphQL, um CSV normalizado e sete arquivos de resultados sem diferenças de nomes, valores ou quantidade de registros.

A presença de valores ausentes exigiu decisões explícitas. Alguns repositórios não possuem linguagem primária identificada pelo GitHub. Outros não possuem nenhuma issue aberta ou fechada, impossibilitando o cálculo do percentual de fechamento. Esses casos não poderiam ser substituídos arbitrariamente por uma linguagem ou por uma taxa igual a zero.

As distribuições de pull requests, releases e dias desde o último push apresentaram grande assimetria. Poucos projetos concentram valores muito superiores aos demais, fazendo com que a média seja fortemente influenciada pelos extremos. Por esse motivo, a mediana e os quartis foram priorizados na interpretação.

A RQ07 apresentou um desafio adicional: as linguagens possuem tamanhos de grupos muito diferentes. Algumas aparecem em centenas de repositórios, enquanto outras possuem apenas uma ou duas observações. As medianas de grupos muito pequenos não oferecem a mesma estabilidade das linguagens mais representadas.

Por fim, a construção do front-end exigiu integrar coleta, transformação, execução das análises, visualização e exportação em um único fluxo interativo.

### 3.2 Tomadas de Decisão

A coleta oficial da S02 foi executada pelo `main.py`, configurado para obter exatamente 1.000 repositórios. Cada consulta solicita até 50 registros e envia, por meio da variável `after`, o cursor obtido na página anterior.

A query solicita os campos hasNextPage e endCursor dentro de pageInfo. Antes de solicitar uma nova página, o coletor verifica se o GitHub informa a existência de resultados adicionais, se o cursor foi retornado e se ele ainda não havia sido utilizado. Uma página vazia ou o encerramento da busca antes dos 1.000 registros provoca uma falha explícita.

Os nomes completos no formato proprietário/repositório são utilizados como chave de unicidade. Assim, registros repetidos não são adicionados novamente à amostra.

A lista de nós extraídos foi armazenada em repositorios\_graphql.json. Esse arquivo preserva os campos dos repositórios, mas não representa o envelope integral da resposta GraphQL, pois não contém data, search, pageInfo, cursores ou informações de limite da API.

Após a coleta, os nós foram transformados em um DataFrame e exportados para repositorios\_populares.csv. O comando utilizado foi:

```
python main.py --csv
```

As decisões de transformação e análise foram:

- Representar a ausência de linguagem primária pela categoria Não definida.
- Calcular idade pela diferença entre o instante da transformação e createdAt, dividida por 365,25.
- Calcular recência pela diferença, em dias, entre o instante da transformação e pushedAt.
- Calcular o percentual de fechamento somente quando a soma de issues abertas e fechadas for maior que zero.
- Manter ausente o percentual dos repositórios que não possuem issues.
- Não remover outliers confirmados nos dados recebidos da API.
- Priorizar mediana e quartis nas distribuições assimétricas.
- Excluir a categoria Não definida da comparação agregada da RQ07.
- Utilizar o Octoverse 2025 como referência fixa para as linguagens do Top 10.
- Carregar o token por meio da variável de ambiente GITHUB\_TOKEN, evitando incorporá-lo ao código.

Os dados oficiais do relatório foram obtidos pelo main.py. O Streamlit foi utilizado como camada interativa e exploratória. Essa separação é importante porque a interface permite escolher outras quantidades e linguagens, podendo gerar amostras diferentes daquela utilizada como snapshot oficial.

### 3.3 Etapas

O processo foi acompanhado no GitHub Project público denominado Laboratorio de Experimentação - Kanban. O fluxo configurado contém as colunas Backlog, To Do, Doing, In review e Done.

| Coluna    | Limite de WIP |
|-----------|---------------|
| Backlog   | 30            |
| Doing     | 9             |
| In review | 5             |

Também foram configurados os campos Priority, Size, Estimate, Start date e Target date. O Project possui visões específicas para backlog, prioridades, itens do time, roadmap e tarefas atribuídas ao usuário atual.

Na S01, Diogo ficou responsável pela consulta automática, RQ01, RQ03, RQ05 e pela primeira versão do Streamlit. Pedro ficou responsável por RQ02, RQ04, RQ06, RQ07 e pelos ajustes de integração.

| Sprint | Atividade                     | Responsável | Issue/PR           |
|--------|-------------------------------|-------------|--------------------|
| S01    | RQ01 — idade                  | Diogo       | Issue #1           |
| S01    | RQ02 — pull requests          | Pedro       | Issue #2 / PR #19  |
| S01    | RQ03 — releases               | Diogo       | Issue #3           |
| S01    | RQ04 — última atualização     | Pedro       | Issue #4 / PR #18  |
| S01    | RQ05 — linguagem primária     | Diogo       | Issue #5           |
| S01    | RQ06 — issues fechadas        | Pedro       | Issue #6 / PR #17  |
| S01    | Consulta automática GraphQL   | Diogo       | Issue #7           |
| S01    | Interface web Streamlit       | Diogo       | Issue #9           |
| S01    | RQ07 — cruzamento de métricas | Pedro       | Issue #16 / PR #20 |
| S01    | Integração geral              | Pedro       | PR #21             |

Na S02, Diogo realizou a paginação, a coleta dos 1.000 repositórios, a exportação CSV e a evolução dos gráficos. Pedro realizou a validação dos dados, a formulação das hipóteses, a atualização metodológica e a preparação do snapshot.

| Sprint | Atividade                          | Responsável | Issue/PR                |
|--------|------------------------------------|-------------|-------------------------|
| S02    | Paginação da API GraphQL           | Diogo       | Issue #8                |
| S02    | Coleta dos 1.000 repositórios      | Diogo       | Issue #10 / PR #27      |
| S02    | Exportação para CSV                | Diogo       | Issue #11 / PR #28      |
| S02    | Validação geral das RQs            | Pedro       | Issue #12               |
| S02    | Hipóteses e validações individuais | Pedro       | Issues #32–#38          |
| S02    | Correção e criação de gráficos     | Diogo       | Issues #29–#30 / PR #31 |
| S02    | Criação das hipóteses              | Pedro       | Issue #13               |
| S02    | Snapshot do GitHub Project         | Pedro       | Issue #14               |
| S02    | Documentação e metodologia         | Pedro/grupo | Issues #15 e #22–#26    |

As hipóteses individuais foram registradas nos comentários das Issues #32 a #38, fechadas após a validação.

### 3.4 Ferramentas

O desenvolvimento foi realizado em Python 3.13.14. Requests 2.34.2 foi utilizada para requisições HTTP à API GraphQL, enquanto python-dotenv 1.2.2 permitiu carregar o token sem incorporá-lo ao código-fonte.

Pandas 3.0.5 foi utilizado na transformação dos nós GraphQL em estrutura tabular, no cálculo das métricas, nos agrupamentos e na exportação. A interface foi construída com Streamlit 1.61.1. Altair 6.2.2 foi empregado nas visualizações interativas.

Git e GitHub foram utilizados no controle de versão. Issues, pull requests e GitHub Projects registraram tarefas, responsáveis e andamento do trabalho.

- `github_collector/schema.py`: definição da query GraphQL.
- `github_collector/client.py`: comunicação utilizada pelo Streamlit.
- `utils/dataframe.py`: transformação dos nós para a tabela analítica.
- `analysis`: módulos independentes de RQ01–RQ07.
- `main.py`: coleta oficial em lote, exportação e análises.
- `app.py`: camada interativa de apresentação.

As versões foram obtidas do ambiente virtual. Como ainda não existe arquivo de dependências versionado, recomenda-se criar um `requirements.txt` para melhorar a reprodução.

### 3.5 Tabela de Métricas

| RQ   | Métrica                   | Definição operacional                                                        | Unidade              | Fonte               |
|------|---------------------------|------------------------------------------------------------------------------|----------------------|---------------------|
| RQ01 | Idade                     | Diferença entre transformação e <code>createdAt</code> , dividida por 365,25 | Anos                 | GraphQL             |
| RQ02 | PRs mescladas             | <code>pullRequests(states: MERGED).totalCount</code>                         | Quantidade           | GraphQL             |
| RQ03 | Releases                  | <code>releases.totalCount</code>                                             | Quantidade acumulada | GraphQL             |
| RQ04 | Recência                  | Diferença entre transformação e <code>pushedAt</code>                        | Dias                 | GraphQL             |
| RQ05 | Linguagem popular         | Pertencimento de <code>primaryLanguage</code> ao Top 10 Octoverse 2025       | Categoria / %        | GraphQL e Octoverse |
| RQ06 | Issues fechadas           | $100 \times \text{fechadas} / (\text{abertas} + \text{fechadas})$            | Percentual           | GraphQL             |
| RQ07 | Linguagem versus métricas | Comparação das medianas de RQ02–RQ04 entre Top 10 e demais                   | Medianas por grupo   | Dados derivados     |

A RQ03 utiliza o termo frequência, mas a implementação mede a quantidade acumulada de releases. Uma frequência exigiria normalização pela idade ou intervalos entre releases. Da mesma maneira, a RQ04 mede recência do último push, não a frequência histórica.

### 3.6 Inovações Propostas pelo Grupo (30% da nota)

As sete questões de pesquisa foram sugeridas pelo professor e não constituem contribuição adicional. A inovação proposta foi uma aplicação web com Streamlit para integrar mineração, processamento, análise, visualização e exportação em um único ambiente.

O Streamlit é um framework Python para construção de aplicações dinâmicas de dados. Sua utilização permitiu transformar os scripts em uma interface navegável, reduzindo a necessidade de interação direta com o terminal.

A aplicação permite informar o token por campo protegido ou variável de ambiente, escolher uma linguagem opcional, solicitar entre 1 e 1.000 repositórios e iniciar a consulta. Uma barra de progresso informa o andamento e mensagens específicas tratam erros HTTP, GraphQL e de conexão.

Depois da coleta, os dados são transformados e as sete análises executadas automaticamente. A interface apresenta indicadores, uma aba por RQ, distribuições, rankings, tabelas, URLs clicáveis, cruzamentos por linguagem e download de CSV enriquecido.

Essa inovação possui caráter de engenharia, transparência e comunicação. Para consistência, o snapshot oficial foi produzido pelo main.py; o Streamlit foi tratado como ferramenta exploratória, pois aceita filtros e quantidades diferentes.

A aplicação representa uma primeira versão. Sua quantidade padrão é 100; o coletor do app.py não possui as mesmas garantias de cardinalidade, unicidade e avanço de cursor do main.py, e sua query não inclui explicitamente is:public. Uma consulta pode sobrescrever o CSV, o JSON normalizado e os resultados das RQs. A interface também anuncia um filtro mínimo de três repositórios por linguagem que o módulo ainda não aplica, e a visualização da RQ07 precisa diferenciar o Top 10 das demais linguagens.

## 4. Resultados

### 4.1 Coleta e validação dos dados

O snapshot validado foi gerado em 20 de agosto de 2026. A transformação ocorreu aproximadamente às 09h49 no horário de Brasília, valor inferido dos artefatos porque o horário exato ainda não é armazenado como metadado.

A coleta produziu exatamente 1.000 repositórios, com 1.000 nomes e 1.000 URLs únicas. A lista de nós GraphQL e o CSV contêm os mesmos projetos e na mesma ordem.

Os repositórios estão organizados pelo número de estrelas de forma não crescente. No instante da coleta, o maior valor foi 541.471 estrelas e o menor foi 32.950.

O CSV possui 1.000 linhas e 11 colunas: Nome, URL, Estrelas, Linguagem, Idade em anos, PRs aceitas, Total de releases, Dias desde a última atualização, Issues abertas, Issues fechadas e Percentual de issues fechadas.

Os campos diretamente coletados foram comparados entre JSON e CSV. Nenhuma divergência foi encontrada, e os sete arquivos de resultados coincidiram com o recálculo realizado a partir do CSV. Esse snapshot foi validado localmente e deve ser versionado antes da entrega para completar sua rastreabilidade.



Foram encontrados 87 repositórios sem linguagem primária, representados por Não definida, e 43 sem issues abertas ou fechadas. Nesses casos, o percentual de fechamento foi mantido ausente. Não foram observados valores negativos, contagens fracionárias, percentuais fora de 0% a 100% ou duplicações.

Os outliers identificados estavam presentes nos dados GraphQL e foram mantidos. Sua remoção reduziria artificialmente a diversidade da amostra.

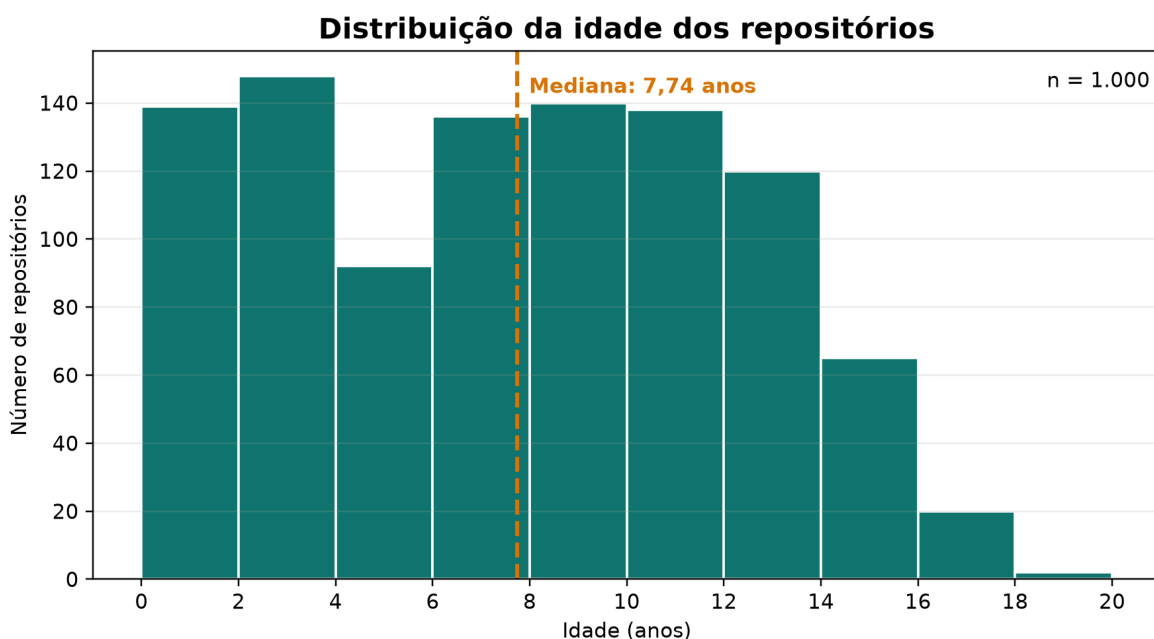
Durante a validação foi identificado um artefato inconsistente denominado `repositorios_populares.json`, com somente 100 registros e valores NaN. Ele foi excluído da análise da S02. Os artefatos canônicos são `repositorios_graphql.json`, `repositorios_populares.csv` e os sete arquivos de RQ.

## 4.2 Visualizações e resultados por RQ

### 4.2.1 RQ01 — Sistemas populares são maduros/antigos?

A idade mediana foi de 7,74 anos e a média foi de 7,66 anos. O mínimo foi 0,02 ano e o máximo 18,36 anos. Dos 1.000 projetos, 139 tinham menos de dois anos, 185 entre dois e menos de cinco, 331 entre cinco e menos de dez e 345 tinham dez anos ou mais.

Assim, 676 repositórios, correspondentes a 67,6% da amostra, possuíam pelo menos cinco anos. Nenhum valor foi classificado como outlier pelo critério de 1,5 vezes a amplitude interquartil.



**Figura 2 — Distribuição da idade dos repositórios.**

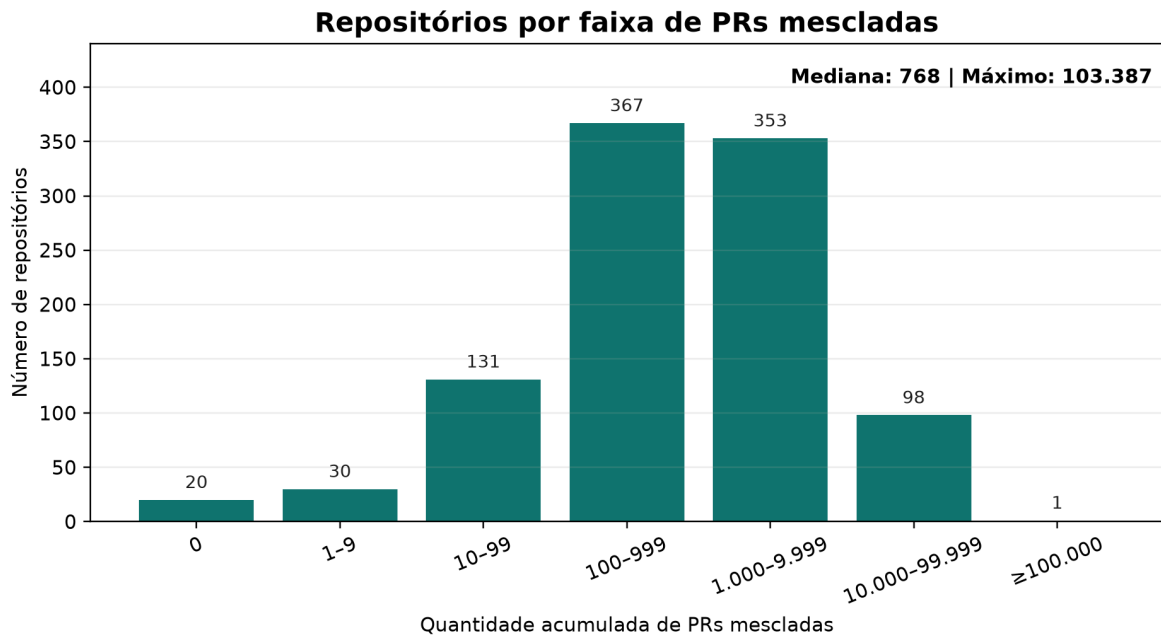
*Fonte: Elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.*

### 4.2.2 RQ02 — Sistemas populares recebem muita contribuição externa?

A mediana foi de 768 pull requests mescladas e a média foi de 4.240,84. O primeiro quartil foi 175, o terceiro 3.423,5 e o máximo 103.387. Vinte repositórios apresentaram zero PRs mescladas.

Foram identificados 124 outliers superiores. A diferença entre média e mediana demonstra forte assimetria, com poucos projetos concentrando uma quantidade muito elevada.

| Repositório                            | PRs mescladas |
|----------------------------------------|---------------|
| firstcontributions/first-contributions | 103.387       |
| llvm/llvm-project                      | 97.254        |
| elastic/elasticsearch                  | 95.619        |



**Figura 3 — Distribuição dos repositórios por faixa de pull requests mescladas.**

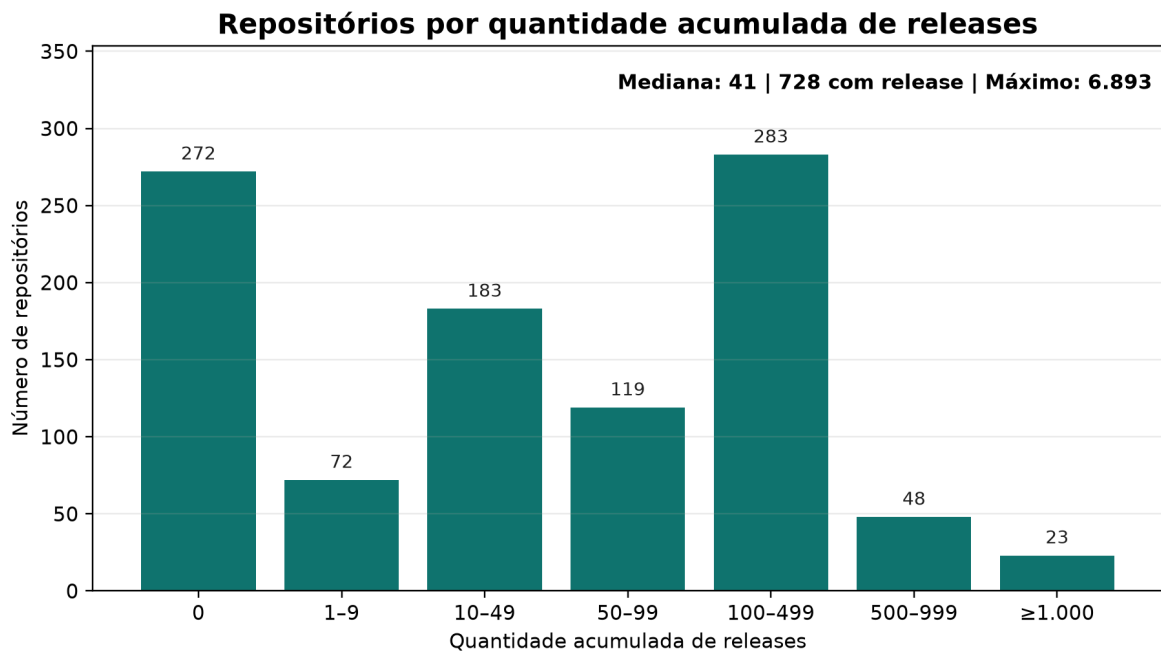
*Fonte: Elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.*

**4.2.3 RQ03 — Sistemas populares lançam releases com frequência?**

A mediana foi de 41 releases e a média foi de 158,23. O primeiro quartil foi zero, o terceiro 153 e o máximo 6.893. Dos 1.000 projetos, 728 possuíam ao menos uma release, representando 72,8% da amostra; 272 apresentaram zero.

Foram identificados 90 outliers superiores, confirmando que poucos projetos acumulam números muito acima da maior parte da amostra.

| Repositório        | Releases |
|--------------------|----------|
| ggml-org/llama.cpp | 6.893    |
| gradio-app/gradio  | 5.090    |
| vercel/next.js     | 3.810    |



**Figura 4 — Distribuição dos repositórios por quantidade acumulada de releases.**

*Fonte: Elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.*

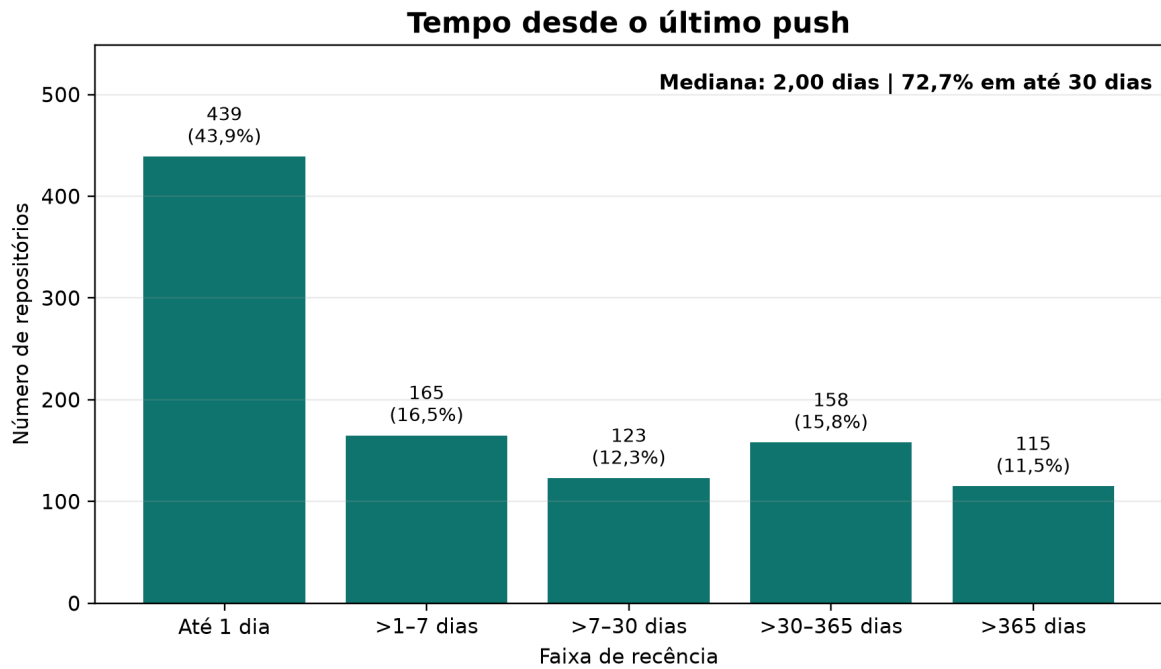
#### 4.2.4 RQ04 — Sistemas populares são atualizados com frequência?

A mediana do tempo desde o último push foi de aproximadamente 2,00 dias. A média foi de 114,05 dias e o máximo chegou a 2.452,35 dias.

| Recência                  | Repositórios |
|---------------------------|--------------|
| Até 1 dia                 | 439          |
| Mais de 1 e até 7 dias    | 165          |
| Mais de 7 e até 30 dias   | 123          |
| Mais de 30 e até 365 dias | 158          |
| Mais de 365 dias          | 115          |

Portanto, 727 repositórios, ou 72,7%, receberam um push nos 30 dias anteriores à transformação. Ao mesmo tempo, 11,5% estavam há mais de um ano sem push. Foram identificados 197 outliers superiores.

Um valor igual a zero foi produzido porque o pushedAt retornado pelo GitHub estava aproximadamente 17 segundos à frente do relógio utilizado. O código limitou o resultado mínimo a zero. Quando pushedAt está ausente, a transformação utiliza createdAt como alternativa; esse fallback não foi acionado nos 1.000 registros desta coleta.



**Figura 5 — Distribuição do tempo desde o último push.**

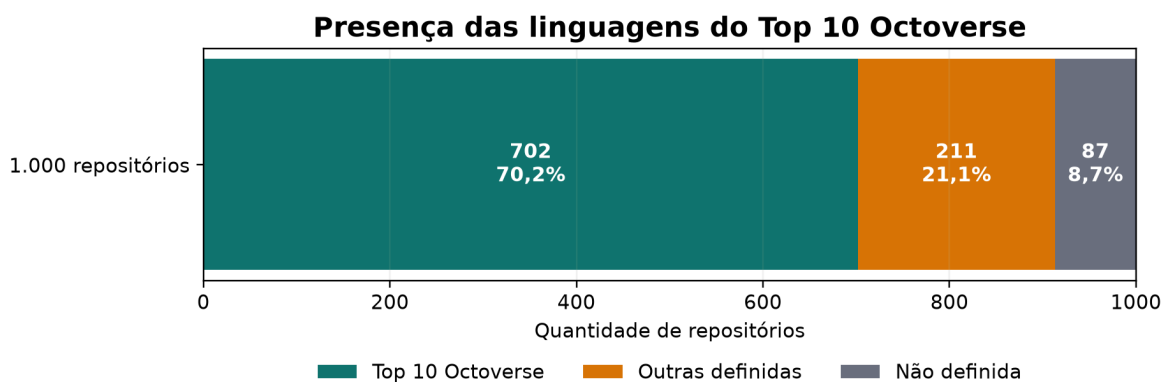
*Fonte: Elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.*

#### 4.2.5 RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Foram identificadas 43 linguagens efetivamente definidas. O resultado apresenta 44 categorias porque inclui Não definida.

| Linguagem    | Repositórios |
|--------------|--------------|
| Python       | 227          |
| TypeScript   | 173          |
| JavaScript   | 111          |
| Não definida | 87           |
| Go           | 77           |
| Rust         | 58           |
| C++          | 41           |
| Java         | 41           |

Python, TypeScript e JavaScript concentram 511 repositórios. Ao todo, 702 projetos utilizam uma linguagem do Top 10, correspondendo a 70,2% da amostra. Considerando somente os 913 com linguagem definida, a proporção é 76,9%. Doze linguagens aparecem em somente um repositório.



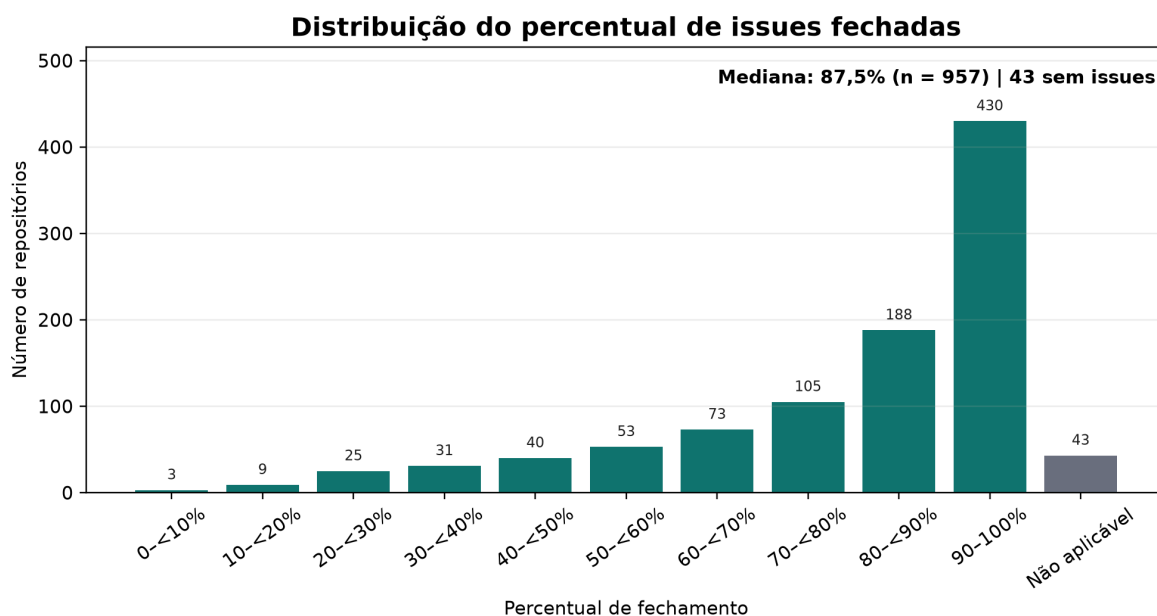
**Figura 6 — Participação das linguagens do Top 10 do GitHub Octoverse.**

Fonte: Elaborado pelo grupo com dados da API GraphQL e do GitHub Octoverse 2025.

#### 4.2.6 RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

O percentual foi calculado para 957 repositórios; nos 43 restantes não existiam issues abertas ou fechadas. Entre os válidos, a mediana foi 87,5%, a média 80,25%, o primeiro quartil 70,4%, o terceiro 96,8%, o mínimo 7,7% e o máximo 100%.

Dos 957 repositórios com issues, 618, equivalentes a 64,6%, apresentaram pelo menos 80% de fechamento. Em contrapartida, 108 projetos, ou 11,3%, possuíam percentual abaixo de 50%. Foram identificados 38 outliers inferiores.



**Figura 7 — Distribuição do percentual de issues fechadas.**

Fonte: Elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

#### 4.2.7 RQ07 — Linguagens populares recebem mais contribuição, releases e atualizações?

Os 913 repositórios com linguagem definida foram divididos entre linguagens do Top 10 e outras linguagens. Os 87 classificados como Não definida foram excluídos. Essa comparação agregada foi

recalculada para o relatório a partir do CSV; o JSON atual da RQ07 armazena as medianas separadas por linguagem e a marcação de pertencimento ao Top 10.

| Grupo                       | N   | Mediana de PRs | Mediana de releases | Mediana de dias |
|-----------------------------|-----|----------------|---------------------|-----------------|
| Linguagens Top 10           | 702 | 1.000          | 64                  | 1,05            |
| Outras linguagens definidas | 211 | 670            | 31                  | 3,67            |

Os projetos associados ao Top 10 apresentaram maior mediana de pull requests e releases e menor número de dias desde o último push, indicando maior recência.

A comparação individual exige cautela: das 44 categorias apresentadas, 24 possuem menos de cinco repositórios, 19 possuem no máximo dois e 12 são representadas por um único projeto.

### Indicadores medianos por grupo de linguagens

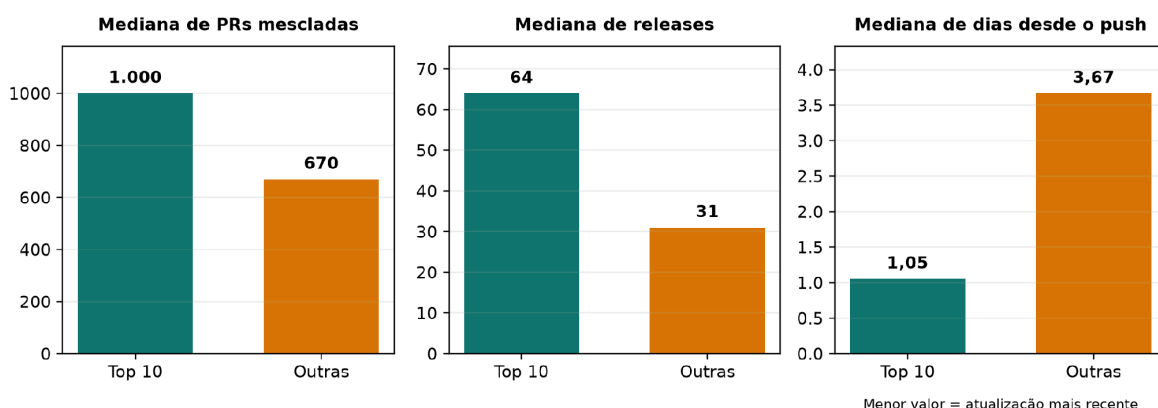


Figura 8 — Indicadores medianos por grupo de linguagens.

Fonte: Elaborado pelo grupo com dados da API GraphQL e do GitHub Octoverse 2025.

## 4.3 Discussão

### 4.3.1 RQ01

O resultado foi compatível com a expectativa informal de que repositórios populares tendem a ser maduros. A mediana de 7,74 anos ultrapassou cinco anos, e 67,6% da amostra possuía cinco anos ou mais. Entretanto, projetos antigos tiveram mais tempo para acumular estrelas; não se pode concluir que maturidade seja consequência da popularidade.

### 4.3.2 RQ02

A mediana de 768 PRs ultrapassou o limiar operacional da expectativa informal. Os 124 outliers e a diferença entre média e mediana são compatíveis com a assimetria esperada. A comparação vale somente para PRs mescladas; a origem externa não pode ser confirmada.

#### 4.3.3 RQ03

O resultado foi compatível com a expectativa quanto à presença e quantidade acumulada: 72,8% possuíam release e a mediana foi 41. O total não mede frequência temporal; projetos antigos tiveram mais tempo para acumular versões e alguns utilizam outros canais.

#### 4.3.4 RQ04

O resultado foi compatível com a expectativa quanto à recência. Um total de 72,7% recebeu push nos 30 dias anteriores e a mediana foi dois dias. A métrica não representa frequência histórica nem distingue atualizações manuais de automáticas.

#### 4.3.5 RQ05

O resultado foi compatível com a expectativa: linguagens Top 10 estavam em 70,2% da amostra e 76,9% dos projetos com linguagem definida. O resultado depende do ranking e de sua data, e a linguagem primária simplifica projetos multilíngues.

#### 4.3.6 RQ06

A mediana de 87,5% ultrapassou o limiar de 80%. Entre os projetos com issues, 64,6% possuíam pelo menos 80% de fechamento. A taxa não informa tempo, complexidade ou qualidade da solução e pode omitir rastreadores externos.

#### 4.3.7 RQ07

O resultado foi compatível com a expectativa informal: o grupo Top 10 apresentou mais PRs, mais releases e menor tempo desde o push. A associação não é causal; idade, domínio, comunidade, governança e automação podem influenciar os resultados.

| Hipótese                                   | Avaliação                                                      |
|--------------------------------------------|----------------------------------------------------------------|
| H01 — idade mediana superior a cinco anos  | Resultado compatível com a expectativa                         |
| H02 — elevado número de contribuições      | Compatível para PRs mescladas; origem externa não identificada |
| H03 — maioria com releases                 | Compatível para presença e quantidade; frequência não medida   |
| H04 — atualização nos últimos 30 dias      | Compatível para recência; frequência histórica não medida      |
| H05 — maioria nas linguagens Top 10        | Resultado compatível com a expectativa                         |
| H06 — mediana de fechamento superior a 80% | Compatível entre os projetos com issues                        |
| H07 — indicadores superiores no Top 10     | Compatível descritivamente, sem evidência causal               |

#### 4.3.8 Ameaças à validade

**Validade de construto.** Estrelas são aproximação de popularidade. PRs mescladas não identificam contribuição externa. Releases acumuladas não medem frequência, dias desde o push medem recência, e linguagem primária não representa toda a composição de projetos multilíngues.

**Validade interna.** O estudo é observacional e não controla idade, domínio, tamanho da equipe ou governança. As relações encontradas não devem ser interpretadas como causais.

**Validade externa.** A amostra contém somente os 1.000 repositórios públicos com mais estrelas retornados pela busca. Os resultados não são automaticamente generalizáveis para repositórios menos populares ou privados.

**Validade de conclusão.** Foram utilizadas estatísticas descritivas, sem testes de significância ou modelos de controle. Grupos pequenos tornam algumas medianas por linguagem instáveis.

**Validade temporal.** Estrelas, PRs, releases, issues e atualizações mudam continuamente. A paginação ocorre em chamadas separadas, durante as quais a ordenação pode variar.

**Confiabilidade e reprodutibilidade.** O instante e os parâmetros não são armazenados como metadados e não existe arquivo de dependências versionado. A interface e o lote utilizam coletores diferentes.

**Limitações da inovação.** O Streamlit não carrega automaticamente o snapshot persistido. Uma nova busca pode sobrescrever arquivos, a quantidade padrão é 100 e a visualização da RQ07 ainda requer correções.



## 5. Conclusão

Este trabalho apresentou uma análise exploratória dos 1.000 repositórios públicos com maior número de estrelas retornados pela API GraphQL do GitHub. A paginação permitiu atingir a quantidade exigida, enquanto a validação confirmou 1.000 nomes e URLs únicos.

Os dados armazenados na lista de nós GraphQL, no CSV e nos arquivos das sete RQs mostraram-se consistentes. Não foram encontradas divergências nos campos diretamente coletados, duplicações ou valores fora de seus domínios esperados.

Os projetos analisados são majoritariamente maduros e apresentam atividade recente. A idade mediana foi 7,74 anos, e 72,7% receberam um push nos 30 dias anteriores.

As distribuições de PRs e releases apresentaram forte assimetria. A mediana foi 768 PRs e 41 releases, mas poucos projetos concentraram valores muito elevados. Por isso, as medianas se mostraram mais representativas que as médias.

As linguagens Top 10 do Octoverse estavam em 70,2% da amostra e 76,9% dos projetos com linguagem definida. Entre os projetos com issues, a mediana de fechamento foi 87,5%.

A RQ07 mostrou que projetos associados ao Top 10 possuíam, descritivamente, mais PRs, mais releases e maior recência. Essa associação não permite concluir causalidade.

Os resultados observados foram compatíveis com as expectativas informais formuladas para as RQs. Essa comparação é exclusivamente exploratória, pois as hipóteses não foram pré-registradas e foram redigidas após uma inspeção preliminar da coleta. A análise não identifica a origem externa, não mede diretamente a frequência de releases e utiliza o último push como aproximação de atualização.

A principal inovação foi a aplicação em Streamlit, que integra parâmetros de mineração, acompanhamento, análises, indicadores, gráficos, inspeção tabular e download do CSV. O laboratório passa a possuir uma camada interativa de comunicação e exploração.

Como trabalhos futuros, recomenda-se:

- Unificar os coletores utilizados pelo main.py e pelo app.py.
- Proteger o snapshot oficial contra sobrescrita por consultas exploratórias.
- Permitir que o Streamlit carregue dados já coletados.
- Armazenar data, horário, query, quantidade, cursores e limite da API como metadados.
- Versionar dependências em requirements.txt.
- Identificar se os autores das PRs são externos.
- Normalizar releases e contribuições pela idade.
- Analisar múltiplos snapshots em diferentes datas.
- Aplicar testes estatísticos e intervalos de confiança.
- Corrigir e ampliar a comparação visual da RQ07.

## 6. Referências

- ALVES, Pedro Henrique Maia; BRUNORO, Diogo C. Lab-experimentacao-software. GitHub, 2026. Disponível em: <https://github.com/PedroMaiaAlves/lab-experimentacao-software>. Acesso em: 20 ago. 2026.
- GITHUB. GraphQL API documentation. 2026a. Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 20 ago. 2026.
- GITHUB. Using pagination in the GraphQL API. 2026b. Disponível em: <https://docs.github.com/en/graphql/guides/using-pagination-in-the-graphql-api>. Acesso em: 20 ago. 2026.
- GITHUB. Queries: search. 2026c. Disponível em: <https://docs.github.com/en/graphql/reference/queries#search>. Acesso em: 20 ago. 2026.
- GITHUB. Octoverse: a new developer joins GitHub every second as AI leads TypeScript to #1. Publicado em 28 out. 2025; atualizado em 28 fev. 2026. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>. Acesso em: 20 ago. 2026.
- KALLIAMVAKOU, E.; GOUSIOS, G.; BLINCOE, K.; SINGER, L.; GERMAN, D. M.; DAMIAN, D. The promises and perils of mining GitHub. In: Proceedings of the 11th Working Conference on Mining Software Repositories. ACM, 2014. p. 92–101. DOI: <https://doi.org/10.1145/2597073.2597074>. Acesso em: 20 ago. 2026.
- PANDAS. Pandas documentation. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 20 ago. 2026.
- PYTHON SOFTWARE FOUNDATION. Python documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 20 ago. 2026.
- REQUESTS. Requests: HTTP for Humans. Disponível em: <https://requests.readthedocs.io/en/latest/>. Acesso em: 20 ago. 2026.
- STREAMLIT. Streamlit documentation. Disponível em: <https://docs.streamlit.io/>. Acesso em: 20 ago. 2026.
- VEGA-ALTAIR. Declarative visualization in Python. Disponível em: <https://altair-viz.github.io/>. Acesso em: 20 ago. 2026.